MANUAL TÉCNICO Y DE OPERACIÓN

Sistema de Gestión de Parqueaderos

Parking Lot Management System

Full-Stack Web Application

Proyecto:	Sistema de Gestión de Parqueaderos
Autores:	Adrian Yesid Restrepo | Oscar David Gamboa | Johan Andrey Forero
Stack tecnológico:	Node.js | Express.js | MySQL | JavaScript ES6 | HTML5 | CSS3
Versión:	1.0
Fecha:	2026

Documento de carácter académico

Tabla de Contenido

1. INTRODUCCIÓN 4
2. OBJETIVOS DEL SISTEMA 4
   2.1 Objetivo General 4
   2.2 Objetivos Específicos 4
3. DESCRIPCIÓN DEL SISTEMA 5
   3.1 Descripción General 5
   3.2 Módulos Principales 5
4. DISEÑO TÉCNICO DEL SISTEMA 6
   4.1 Esquema de Requerimientos 6
      4.1.1 Requerimientos Funcionales 6
      4.1.2 Requerimientos No Funcionales 6
   4.2 Software Base y Prerrequisitos 6
      4.2.1 Requerimientos de Hardware (Mínimos) 6
      4.2.2 Requerimientos de Software 7
   4.3 Componentes y Estándares 7
      4.3.1 Backend – Dependencias 7
      4.3.2 Frontend – Tecnologías 7
      4.3.3 Patrones de Diseño Implementados 8
      4.3.4 Estándares de Codificación 8
5. MODELO DE DATOS 9
   5.1 Descripción de Entidades 9
   5.2 Diccionario de Datos – Tablas Principales 9
      5.2.1 Tabla: usuarios 9
      5.2.2 Tabla: perfiles 10
      5.2.3 Tabla: vehículos 10
      5.2.4 Tabla: Celdas 10
      5.2.5 Tabla: accesos_salidas 10
      5.2.6 Tabla: incidencias 11
      5.2.7 Tabla: reporte_incidencias 11
      5.2.8 Tabla: pico_y_placa 11
6. FUNCIONALIDAD Y SERVICIOS OFRECIDOS 12
   6.1 API RESTful – Endpoints 12
      6.1.1 Usuarios 12
      6.1.2 Vehículos 12
      6.1.3 Celdas 12

6.1.4 Accesos y Salidas 13
6.1.5 Incidencias 13
6.1.6 Pico y Placa 13
6.2 Flujo de Autenticación 13
6.3 Módulo de Reportes 14
7. ORGANIZACIÓN DE COMPONENTES Y ARQUITECTURA 15
7.1 Estructura del Proyecto 15
7.2 Diagrama de Flujo de Datos 15
7.3 Configuración de Puertos y Comunicación 16
8. INSTALACIÓN Y DESPLIEGUE 17
8.1 Prerrequisitos de Instalación 17
8.2 Script de Instalación Paso a Paso 17
Paso 1: Clonar el repositorio 17
Paso 2: Instalar dependencias del backend 17
Paso 3: Crear y configurar la base de datos 17
Paso 4: Configurar variables de entorno 17
Paso 5: Iniciar el servidor backend 17
Paso 6: Iniciar el frontend 17
8.3 Configuración de Seguridad 18
9. PRUEBAS DEL SISTEMA 19
9.1 Estrategia de Pruebas 19
9.2 Comandos de Prueba 19
10. RESOLUCIÓN DE PROBLEMAS 20
10.1 Errores Técnicos Más Comunes 20
10.2 Logs y Diagnóstico 21
11. MEJORAS FUTURAS Y PENDIENTES 22
11.1 Pendientes 22
11.2 Mejoras Recomendadas para Producción 22

1. INTRODUCCIÓN

El presente Manual Técnico y de Operación describe la arquitectura, diseño, instalación, configuración y operación del Sistema de Gestión de Parqueaderos (Parking Lot Management System). Este documento está orientado a desarrolladores, administradores de sistemas e ingenieros de soporte encargados de mantener y operar la solución.

El sistema provee una plataforma web Full-Stack que permite gestionar de forma integral las operaciones de un parqueadero: administración de usuarios y vehículos, control de celdas de parqueo, registro de accesos y salidas, gestión de incidencias, control de pico y placa, y generación de reportes. Fue construido siguiendo patrones de diseño modernos con una arquitectura cliente-servidor bien definida.

2. OBJETIVOS DEL SISTEMA

2.1 Objetivo General

Desarrollar e implementar un sistema de información web completo para la gestión integral de un parqueadero, que permita controlar el acceso vehicular, administrar recursos, gestionar usuarios y generar reportes operativos mediante una arquitectura Full-Stack robusta y segura.

2.2 Objetivos Específicos

- Implementar un sistema de autenticación con control de acceso basado en roles (Administrador, Operario, Usuario).
- Gestionar el registro y seguimiento de vehículos asociados a usuarios del sistema.
- Controlar en tiempo real la disponibilidad de celdas de parqueo mediante representación visual dinámica.
- Registrar y calcular automáticamente los tiempos de estadía en cada acceso vehicular.
- Integrar validaciones de pico y placa según el tipo de vehículo y día de la semana.
- Gestionar incidencias asociadas a vehículos dentro del parqueadero.
- Generar reportes en 10 categorías distintas con filtros y exportación a PDF.
- Proveer documentación técnica completa mediante Swagger/OpenAPI.

3. DESCRIPCIÓN DEL SISTEMA

3.1 Descripción General

El Sistema de Gestión de Parqueaderos es una aplicación web Full-Stack desarrollada con Node.js y Express en el backend, y HTML5/CSS3/JavaScript puro en el frontend. Se conecta a una base de datos relacional MySQL y expone una API RESTful completamente documentada con Swagger.

El sistema cuenta con 9 vistas funcionales que cubren todos los procesos del parqueadero, con un diseño unificado que incluye header, navegación, footer y sistema de notificaciones. El control de acceso por roles asegura que cada tipo de usuario solo pueda acceder a las funcionalidades correspondientes a su perfil.

3.2 Módulos Principales

Módulo	Descripción	Roles con acceso
Login	Autenticación diferenciada por tipo de usuario	Todos
Perfil de Usuario	Visualización de datos personales y vehículos asociados	Todos
Gestión de Usuarios	CRUD completo de usuarios del sistema	Administrador
Registro de Vehículos	CRUD de vehículos vinculados a usuarios	Admin / Operario
Celdas de Parqueo	Visualización y gestión del estado de celdas	Admin / Operario
Accesos y Salidas	Registro de entrada/salida con cálculo de estadía	Admin / Operario
Incidencias	CRUD de tipos de incidencia y reporte de eventos	Admin / Operario
Pico y Placa	Gestión de restricciones vehiculares	Admin / Operario
Reportes	10 tipos de reporte con filtros y exportación PDF	Administrador

4. DISEÑO TÉCNICO DEL SISTEMA

4.1 Esquema de Requerimientos

4.1.1 Requerimientos Funcionales

- RF01 – Autenticación: El sistema debe autenticar usuarios según su rol. Admin/Operario requieren documento + contraseña (bcrypt); Usuario solo documento.
- RF02 – Gestión de Usuarios: CRUD completo con validación de campos y hashing de contraseñas.
- RF03 – Gestión de Vehículos: Registro, actualización y eliminación de vehículos vinculados a usuarios.
- RF04 – Control de Celdas: Visualización en grid del estado de cada celda (disponible/ocupada) con información del vehículo.
- RF05 – Registro de Accesos: Registro de entrada y salida con validación de pico y placa y cálculo automático de estadía.
- RF06 – Gestión de Incidencias: CRUD de tipos y reporte de incidencias por vehículo.
- RF07 – Pico y Placa: CRUD de restricciones por día y tipo de vehículo.
- RF08 – Reportes: Generación de 10 tipos de reportes con filtros y exportación a PDF.
- RF09 – Control de Acceso por Roles: Cada vista valida la sesión y el perfil del usuario antes de mostrar contenido.

4.1.2 Requerimientos No Funcionales

- RNF01 – Seguridad: Hashing de contraseñas con bcrypt, validación de sesión en cada vista, CORS configurado.
- RNF02 – Usabilidad: Interfaz consistente con notificaciones slide-in auto-dismiss, retroalimentación visual inmediata.
- RNF03 – Disponibilidad: Sistema diseñado para funcionamiento continuo con pool de conexiones MySQL.
- RNF04 – Mantenibilidad: Arquitectura modular con separación de responsabilidades (MVC adaptado). Código documentado.
- RNF05 – Escalabilidad: Endpoints RESTful estándares que permiten escalar horizontalmente.
- RNF06 – Compatibilidad: Compatible con navegadores modernos (Chrome, Firefox, Edge). Frontend responsive para desktop.

4.2 Software Base y Prerrequisitos

4.2.1 Requerimientos de Hardware (Mínimos)

Componente	Requerimiento Mínimo	Recomendado
CPU	2 núcleos @ 1.8 GHz	4 núcleos @ 2.5 GHz
Memoria RAM	2 GB	4 GB
Almacenamiento	10 GB libres (HDD)	20 GB libres (SSD)

Red	100 Mbps	1 Gbps

4.2.2 Requerimientos de Software

Software	Versión Mínima	Propósito
Node.js	>= 18.x LTS	Runtime de JavaScript del servidor
MySQL	>= 8.0	Sistema de gestión de base de datos
Git	>= 2.x	Control de versiones y clonación del repositorio
VS Code (recomendado)	Última versión	IDE de desarrollo con extensión Live Server
Navegador web	Chrome 90+ / Firefox 88+ / Edge 90+	Visualización del frontend
npm	>= 8.x (incluido con Node.js)	Gestor de paquetes de Node.js

4.3 Componentes y Estándares

4.3.1 Backend – Dependencias

Librería / Framework	Versión	Propósito
Express.js	^5.2.1	Framework HTTP para la API REST
mysql2	^3.20.0	Conector MySQL con soporte para pool de conexiones
bcrypt	^6.0.0	Hashing seguro de contraseñas
swagger-jsdoc	^6.2.8	Generación de documentación OpenAPI desde JSDoc
swagger-ui-express	^5.x	Interfaz visual para la documentación Swagger
Jest	^30.3.0	Framework de pruebas unitarias
Nodemon	^3.1.14	Reinicio automático del servidor en desarrollo
cors	^2.x	Middleware para configuración de CORS
dotenv	^16.x	Carga de variables de entorno desde archivo .env

4.3.2 Frontend – Tecnologías

Tecnología	Versión / Estándar	Propósito
HTML5	Estándar W3C	Estructura semántica de las vistas
CSS3	Grid / Flexbox	Estilos y layouts responsivos (desktop)
JavaScript ES6+	ES Modules (import/export)	Lógica del cliente modular

Fetch API	Nativa del navegador	Comunicación asíncrona con la API REST
sessionStorage	Web Storage API	Gestión de sesión de usuario en cliente
Poppins (font)	Google Fonts	Tipografía principal de la interfaz

4.3.3 Patrones de Diseño Implementados

- MVC Adaptado: Separación en modelos (clases con encapsulación), servicios (lógica de negocio), rutas (controladores) y vistas (frontend).
- Repository Pattern: Los servicios abstraen el acceso a datos, separando la lógica de negocio de las consultas SQL.
- Module Pattern (ES6): El frontend usa imports/exports para organizar la lógica por responsabilidad.
- Singleton (Pool de conexiones): El pool de conexiones MySQL se instancia una sola vez y se reutiliza en toda la aplicación.
- Factory Pattern: Las clases de modelos actúan como factories para la creación de entidades del dominio.

4.3.4 Estándares de Codificación

- Nombres de variables y funciones: camelCase.
- Nombres de clases y constructores: PascalCase.
- Nombres de archivos: snake_case para módulos JS del frontend, camelCase para archivos del backend.
- Atributos de clase: prefijo # para campos privados (encapsulación ES2022).
- Comentarios JSDoc en modelos y servicios del backend.
- Constantes de configuración centralizadas en archivos de config/.

5. MODELO DE DATOS

5.1 Descripción de Entidades

La base de datos se denomina ParkingLot y contiene las siguientes entidades principales:

Entidad / Tabla	Descripción	Relaciones
usuarios	Almacena los datos de todos los usuarios del sistema (Admin, Operario, Usuario)	FK: perfil_id → perfiles
perfiles	Catálogo de roles: Administrador (1), Operario (2), Usuario (3)	Referenciada por usuarios
vehiculos	Registro de vehículos con placa, tipo y propietario	FK: usuario_id → usuarios
celdas	Celdas del parqueadero con tipo y estado de disponibilidad	Referenciada por accesos_salidas
accesos_salidas	Registro de cada movimiento de entrada/salida vehicular	FK: vehiculo_id, celda_id
incidencias	Catálogo de tipos de incidencia posibles	Referenciada por reporte_incidencias
reporte_incidencias	Relación N:M entre vehículos e incidencias con fecha y descripción	FK: vehiculo_id, incidencia_id
pico_y_placa	Restricciones de circulación por día y tipo de vehículo	Independiente

5.2 Diccionario de Datos – Tablas Principales

5.2.1 Tabla: usuarios

Campo	Tipo	Restricciones	Descripción
id_usuario	INT	PK, AUTO_INCREMENT	Identificador único del usuario
nombre	VARCHAR(100)	NOT NULL	Nombre completo del usuario
documento	VARCHAR(20)	NOT NULL, UNIQUE	Número de documento de identidad
correo	VARCHAR(100)	UNIQUE	Correo electrónico del usuario
contrasena	VARCHAR(255)	NULL	Hash bcrypt de la contraseña (solo admin/operario)
perfil_id	INT	FK → perfiles	Rol del usuario en el sistema
fecha_registro	DATETIME	DEFAULT NOW()	Fecha y hora de creación del registro

5.2.2 Tabla: perfiles

Campo	Tipo	Restricciones	Descripción
id_vehiculo	INT	PK, AUTO_INCREMENT	Identificador del perfil
id_perfil	INT	PK, AUTO_INCREMENT	Identificador del perfil
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol
descripcion	VARCHAR(100)	NULL	Descripción del perfil

5.2.3 Tabla: vehículos

Campo	Tipo	Restricciones	Descripción
id_vehiculo	INT	PK, AUTO_INCREMENT	Identificador único del vehículo
placa	VARCHAR(10)	NOT NULL, UNIQUE	Placa del vehículo
tipo_vehiculo	VARCHAR(50)	NOT NULL	Tipo (carro, moto, camión, etc.)
modelo	VARCHAR(100)	NULL	Modelo o descripción del vehículo
usuario_id	INT	FK → usuarios	Propietario del vehículo

5.2.4 Tabla: Celdas

Campo	Tipo	Restricciones	Descripción
Campo	Tipo	Restricciones	Descripción
id_celda	INT	PK, AUTO_INCREMENT	Identificador de la celda
numero_celda	VARCHAR(10)	NOT NULL, UNIQUE	Número de celda
tipo_celda	VARCHAR(50)	NOT NULL	Tipo de celda
estado	ENUM	DEFAULT 'disponible'	Estado de la celda
descripcion	VARCHAR(100)	NULL	Descripción adicional

5.2.5 Tabla: accesos_salidas

Campo	Tipo	Restricciones	Descripción
id_acceso	INT	PK, AUTO_INCREMENT	Identificador del movimiento
vehiculo_id	INT	FK → vehiculos	Vehículo que ingresa/sale
celda_id	INT	FK → celdas	Celda de parqueo asignada
tipo_movimiento	ENUM	('entrada','salida')	Tipo de movimiento registrado

fecha_hora	DATETIME	NOT NULL	Fecha y hora del movimiento
tiempo_estadia	TIME	NULL	Duración calculada de la estadía

5.2.6 Tabla: incidencias

Campo	Tipo	Restricciones	Descripción
id_incidencia	INT	PK, AUTO_INCREMENT	Identificador
nombre	VARCHAR(100)	NOT NULL	Nombre de la incidencia
descripcion	VARCHAR(255)	NULL	Descripción

5.2.7 Tabla: reporte_incidencias

Campo	Tipo	Restricciones	Descripción
vehiculo_id	INT	FK	Referencia a vehiculos
incidencia_id	INT	FK	Referencia a incidencias
fecha	DATETIME	NOT NULL	Fecha del evento
descripcion	VARCHAR(255)	NULL	Descripción del evento
vehiculo_id	INT	FK	Referencia a vehiculos

5.2.8 Tabla: pico_y_placa

Campo	Tipo	Restricciones	Descripción
id_restriccion	INT	PK, AUTO_INCREMENT	Identificador
dia_semana	VARCHAR(20)	NOT NULL	Día de la semana
tipo_vehiculo	VARCHAR(50)	NOT NULL	Tipo de vehículo
numero_restriccion	VARCHAR(10)	NOT NULL	Restricción de placa
descripcion	VARCHAR(100)	NULL	Descripción adicional

6. FUNCIONALIDAD Y SERVICIOS OFRECIDOS

6.1 API RESTful – Endpoints

La API está disponible en http://localhost:3000 y documentada interactivamente en http://localhost:3000/api-docs (Swagger UI). A continuación se listan todos los endpoints disponibles:

6.1.1 Usuarios

Método	Ruta	Descripción
POST	/api/usuarios/login	Autenticación de usuario según su rol
GET	/api/usuarios	Listar todos los usuarios del sistema
GET	/api/usuarios/:id	Obtener un usuario por su ID
GET	/api/usuarios/perfil	Obtener perfil del usuario en sesión
POST	/api/usuarios	Crear un nuevo usuario
PUT	/api/usuarios/:id	Actualizar datos de un usuario
DELETE	/api/usuarios/:id	Eliminar un usuario del sistema

6.1.2 Vehículos

Método	Ruta	Descripción
GET	/api/vehiculos	Listar todos los vehículos registrados
GET	/api/vehiculos/:id	Obtener un vehículo por su ID
GET	/api/vehiculos/usuario/:usuarioId	Listar vehículos de un usuario específico
POST	/api/vehiculos	Registrar un nuevo vehículo
PUT	/api/vehiculos/:id	Actualizar datos de un vehículo
DELETE	/api/vehiculos/:id	Eliminar un vehículo del sistema

6.1.3 Celdas

Método	Ruta	Descripción
GET	/api/celdas	Listar todas las celdas de parqueo
GET	/api/celdas/:id	Obtener celda por ID con su estado actual
POST	/api/celdas	Crear una nueva celda de parqueo
PUT	/api/celdas/:id	Actualizar estado o datos de una celda
DELETE	/api/celdas/:id	Eliminar una celda del sistema

6.1.4 Accesos y Salidas

Método	Ruta	Descripción
GET	/api/accesos-salidas	Listar todos los movimientos registrados
GET	/api/accesos-salidas/:id	Obtener un movimiento por ID
POST	/api/accesos-salidas	Registrar una nueva entrada o salida
PUT	/api/accesos-salidas/:id	Actualizar un movimiento registrado
DELETE	/api/accesos-salidas/:id	Eliminar un registro de movimiento

6.1.5 Incidencias

Método	Ruta	Descripción
GET	/api/incidencias	Listar tipos de incidencia
GET	/api/incidencias/:id	Obtener tipo de incidencia por ID
POST	/api/incidencias	Crear nuevo tipo de incidencia
PUT	/api/incidencias/:id	Actualizar tipo de incidencia
DELETE	/api/incidencias/:id	Eliminar tipo de incidencia
GET	/api/reporte-incidencias	Listar todos los reportes de incidencia
POST	/api/reporte-incidencias	Crear reporte de incidencia de un vehículo
DELETE	/api/reporte-incidencias/vehiculo/:vId/incidencia/:iId	Eliminar reporte por clave compuesta

6.1.6 Pico y Placa

Método	Ruta	Descripción
GET	/api/pico-y-placa	Listar todas las restricciones de pico y placa
GET	/api/pico-y-placa/:id	Obtener restricción por ID
POST	/api/pico-y-placa	Crear nueva restricción
PUT	/api/pico-y-placa/:id	Actualizar restricción existente
DELETE	/api/pico-y-placa/:id	Eliminar restricción

6.2 Flujo de Autenticación

El sistema implementa autenticación diferenciada según el rol del usuario:

Rol	Perfil ID	Credenciales requeridas	Vistas accesibles

Administrador	1	Documento + Contraseña (bcrypt)	Todas, incluida gestión de usuarios
Operario	2	Documento + Contraseña (bcrypt)	Todas excepto reportes completos
Usuario	3	Solo Documento	Solo perfil personal

Al autenticarse exitosamente, los datos del usuario se almacenan en sessionStorage del navegador. Cada vista del frontend ejecuta la función restricUserAccess() al cargar, que verifica la existencia y validez de la sesión y el rol correspondiente.

6.3 Módulo de Reportes

El módulo de reportes ofrece 10 tipos de informes configurables:

Reporte	Endpoint fuente	Filtros disponibles
Reporte de Usuarios	GET /api/usuarios	Por ID o número de documento
Reporte de Vehículos	GET /api/vehiculos	Por ID o número de placa
Reporte de Entradas	GET /api/accesos-salidas	Por fecha y/o ID de vehículo
Reporte de Salidas	GET /api/accesos-salidas	Por fecha y/o ID de vehículo
Reporte de Incidencias	GET /api/reporte-incidencias	Por fecha y/o ID de vehículo
Reporte Pico y Placa	GET /api/pico-y-placa	Por día o tipo de vehículo
Celdas Ocupadas	GET /api/celdas	Sin filtro (estado actual)
Estado de Celdas	GET /api/celdas	Por tipo o estado de celda
Vehículos Frecuentes	GET /api/accesos-salidas + /api/vehiculos	Por número de placa
Horas Pico	GET /api/accesos-salidas	Por fecha

7. ORGANIZACIÓN DE COMPONENTES Y ARQUITECTURA

7.1 Estructura del Proyecto

El proyecto sigue una arquitectura de capas bien definida con separación total entre frontend y backend:

Capa / Directorio	Responsabilidad	Tecnología principal
backend/app.js	Punto de entrada del servidor, configuración de middlewares	Express.js
backend/config/	Configuración de BD, variables de entorno, script SQL	dotenv, mysql2
backend/models/	Clases del dominio con encapsulación (getters/setters, campos #privados)	JavaScript ES2022
backend/services/	Lógica de negocio y acceso a datos (consultas SQL)	mysql2 pool
backend/routes/	Definición de endpoints REST, validaciones de entrada	Express Router
backend/test/	Pruebas unitarias de modelos y servicios	Jest
frontend/views/	Páginas HTML de cada módulo del sistema	HTML5 semántico
frontend/assets/css/	Hojas de estilo por vista con diseño consistente	CSS3 Grid/Flexbox
frontend/assets/js/	Módulos ES6 con lógica del cliente	JavaScript ES Modules
frontend/assets/images/	Recursos gráficos e iconos de la interfaz	PNG/SVG

7.2 Diagrama de Flujo de Datos

El flujo de datos del sistema sigue el siguiente patrón para cada operación:

- 1. El usuario interactúa con la interfaz HTML del frontend (browser).
- 2. El módulo JavaScript realiza una petición HTTP a la API REST mediante Fetch API.
- 3. Express.js recibe la petición, la enruta al controlador (router) correspondiente.
- 4. El servicio ejecuta la lógica de negocio y consulta la base de datos vía pool de conexiones MySQL2.
- 5. El resultado se serializa a JSON y se devuelve al frontend.
- 6. El módulo JS actualiza el DOM con los datos recibidos.

7.3 Configuración de Puertos y Comunicación

Componente	Puerto por defecto	Protocolo	Configuración
Backend API	3000	HTTP	Variable PORT en archivo .env
Frontend (Live Server)	5500	HTTP	Extensión Live Server de VS Code
MySQL Server	3306	TCP/IP	Variables DB_HOST, DB_PORT en .env
Swagger UI	3000/api-docs	HTTP	Montado sobre el servidor Express

8. INSTALACIÓN Y DESPLIEGUE

8.1 Prerrequisitos de Instalación

Antes de instalar el sistema, verificar que los siguientes componentes estén disponibles en el servidor:

- Node.js >= 18.x LTS instalado y accesible desde línea de comandos (node --version).
- MySQL Server >= 8.0 en ejecución con un usuario con permisos de creación de bases de datos.
- Git >= 2.x para clonar el repositorio.
- npm >= 8.x (se instala junto con Node.js).
- VS Code con la extensión Live Server instalada (para servir el frontend en desarrollo).

8.2 Script de Instalación Paso a Paso

Paso 1: Clonar el repositorio

Ejecutar en la terminal:

```
git clone https://github.com/4DR14N-DEV/Parkin_lot_project.git
cd Parkin_lot_project
```

Paso 2: Instalar dependencias del backend

```
cd backend && npm install
```

Paso 3: Crear y configurar la base de datos

```
mysql -u root -p -e "CREATE DATABASE ParkingLot;"
mysql -u root -p ParkingLot < backend/config/parkingLot.sql
```

Paso 4: Configurar variables de entorno

Crear el archivo .env en la raíz del proyecto con el siguiente contenido:

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_password
DB_NAME=ParkingLot
```

Paso 5: Iniciar el servidor backend

```
cd backend && npm start
```

El servidor estará disponible en: http://localhost:3000

La documentación Swagger estará en: http://localhost:3000/api-docs

Paso 6: Iniciar el frontend

Abrir la carpeta frontend en VS Code, clic derecho sobre el archivo index.html y seleccionar "Open with Live Server". El frontend quedará disponible en: http://localhost:5500

8.3 Configuración de Seguridad

- CORS: Configurado para permitir peticiones desde localhost:5500 durante desarrollo. En producción, ajustar a los dominios autorizados.
- Contraseñas: Hasheadas con bcrypt (factor de costo 10 por defecto). Nunca se almacenan en texto plano.
- Variables de entorno: El archivo .env no debe ser incluido en el repositorio Git (.gitignore). Contiene credenciales sensibles.
- Sesión: Almacenada en sessionStorage (se limpia al cerrar el navegador). Para producción se recomienda migrar a JWT.

9. PRUEBAS DEL SISTEMA

9.1 Estrategia de Pruebas

El sistema implementa pruebas unitarias con Jest, cubriendo los modelos de datos y los servicios de acceso a la base de datos. Los tests se ubican en el directorio backend/test/.

9.2 Comandos de Prueba

Comando	Descripción
npm test	Ejecutar todas las pruebas unitarias
npm run test:coverage	Ejecutar pruebas con reporte de cobertura de código
npm run test:models	Ejecutar únicamente las pruebas de modelos
npm run test:services	Ejecutar únicamente las pruebas de servicios
npm run test:watch	Ejecutar pruebas en modo observación (re-ejecuta al guardar)

10. RESOLUCIÓN DE PROBLEMAS

10.1 Errores Técnicos Más Comunes

Error	Síntomas	Causa probable	Solución
Error de conexión a BD: ECONNREFUSED	El servidor no responde al iniciar	MySQL no está en ejecución o las credenciales en .env son incorrectas	Verificar que MySQL esté activo (service mysql status). Revisar DB_HOST, DB_USER, DB_PASSWORD en .env.
Error 404 en la API	Las peticiones del frontend retornan 404	El backend no está en ejecución o el puerto es incorrecto	Verificar npm start en la carpeta backend. Confirmar PORT=3000 en .env.
CORS Error en el browser	El navegador bloquea peticiones al backend	El frontend se sirve en un puerto diferente al configurado en CORS	Verificar que el frontend corra en puerto 5500. Ajustar la configuración CORS en app.js si es necesario.
Error al ejecutar script SQL	mysql retorna error al importar parkingLot.sql	La base de datos ParkingLot no existe o el usuario no tiene permisos	Crear la BD manualmente: CREATE DATABASE ParkingLot; y otorgar permisos al usuario.
Contraseña no válida (admin/operario)	Login retorna error de credenciales	La contraseña no está hasheada correctamente en la BD	Verificar que el campo contrasena contenga el hash bcrypt, no texto plano.
sessionStorage vacío al recargar	El usuario es redirigido al login al recargar	sessionStorage se limpia al cerrar/recargar la pestaña en algunos browsers	Es comportamiento esperado en esta versión. Volver a iniciar sesión.
Módulo no encontrado (Node.js)	Error: Cannot find module al iniciar el servidor	npm install no se ejecutó o las dependencias están incompletas	Ejecutar npm install en la carpeta backend y verificar que node_modules exista.

10.2 Logs y Diagnóstico

- Los errores del servidor Express se muestran en la consola donde se ejecutó npm start.
- Los errores de MySQL se capturan en los servicios y se propagan como respuesta HTTP con código de error.
- Para depuración avanzada del frontend, usar las Herramientas de Desarrollador del navegador (F12 → Console / Network).
- La documentación Swagger en /api-docs permite probar los endpoints directamente y ver las respuestas en tiempo real.

11. MEJORAS FUTURAS Y PENDIENTES

11.1 Pendientes

- Diseño responsive: Implementar media queries para tablets y dispositivos móviles.

11.2 Mejoras Recomendadas para Producción

- Autenticación JWT: Reemplazar sessionStorage por tokens JWT con refresh tokens para sesiones persistentes y seguras.
- Middleware de validación: Implementar validación de entrada en todas las rutas del backend con librerías como Joi o express-validator.
- Paginación: Agregar paginación en todos los endpoints de listado para manejar grandes volúmenes de datos.
- Rate Limiting: Implementar límite de peticiones por IP para protección contra ataques de fuerza bruta.
- HTTPS: Configurar certificado SSL/TLS para comunicaciones cifradas en producción.
- Logs centralizados: Implementar sistema de logging (Winston o Morgan) para trazabilidad de operaciones.
- Variables de entorno en producción: Usar un gestor de secretos (AWS Secrets Manager, HashiCorp Vault) en lugar de archivos .env.